Multi-Thread Programming-II

- review of threads and race conditions,
- parallel computation,
  - parallelizing problems
  - data and task parallelism,
- thread pools,
- coroutines,
- green threads, Goroutines

Concurrency and parallel computing content and images are copied from [POSIX Threads Programming | LLNL HPC Tutorials]

Review: concurrency

In computer science, concurrency is the ability of different parts or units of a program, algorithm, or problem to be executed out-of-order or in partial order, without affecting the final outcome.

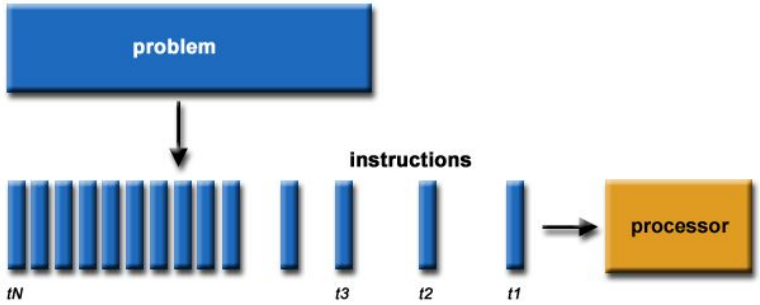
- Operations are **concurrent** if they can progress independently at the same time
- Computers run many programs at the same time.
- Even a single application may be expected to do multiple things at a time.
  - A word processor program is expected to respond keyboard, mouse events at the same time.
- Software that can do multiple things at the same time are called **concurrent software**

Parallel computing

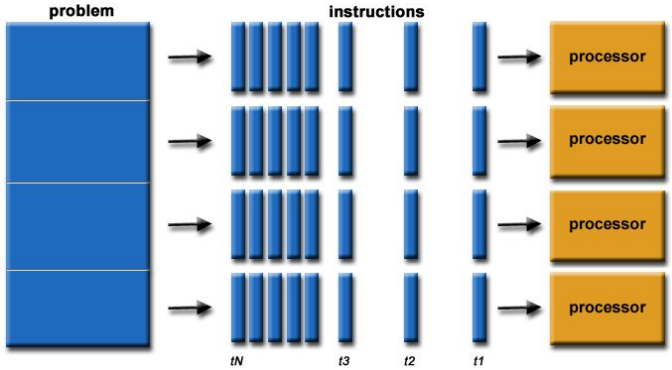
Parallelism: splitting tasks into subtasks

Parallel computing: running subtasks (concurrent parts) parallel in multiple processor

Serial computing



Parallel computing



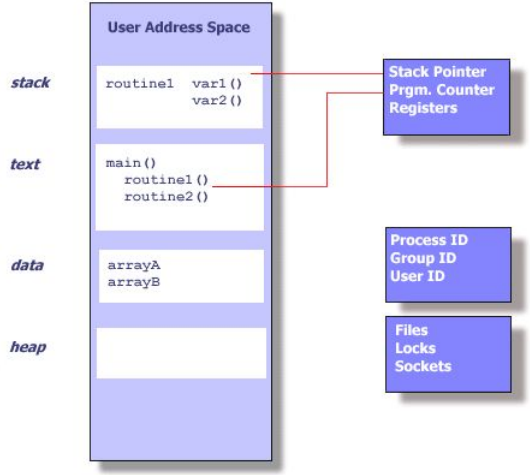
Thread

- Lightweight processes are called threads (software thread).
- A thread is a sequence of computation that can run in parallel with other threads.
- Every program has at least one thread of computation.
  - Main thread
- A program with two or more threads is generally referred to as a **multithreaded program**.
- A multithreaded program is generally used to speed up the solution to a problem.

Processes vs Threads

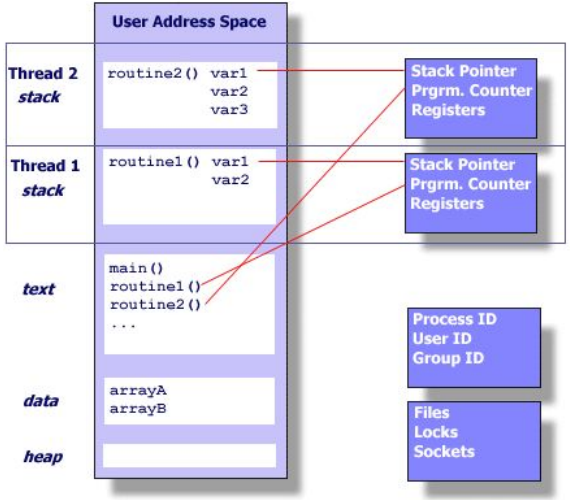
Process

- Each process has its own virtual address space
- fork() is expensive



Thread maintains only its own Stack pointer,Registers, Scheduling properties (such as policy or priority), Set of pending and blocked signals,Thread specific data.

- pthread library allocates some stack space and uses the `clone` function call to start the thread at that stack address.



Creating separate processes is useful

- When more security is desired. For example, Chrome browser uses different processes for different tabs.
- When running an existing and complete program then a new process is required, for example starting ‘gcc’.
- When you are running into synchronization primitives and each process is operating on something in the system.
- When you have too many threads – the kernel tries to schedule all the threads near each other which could cause more harm than good.
- When you don’t want to worry about race conditions
- If one thread blocks in a task (say IO) then all threads block. Processes don’t have that same restriction.
- When the amount of communication is minimal enough that simple IPC needs to be used.

Creating threads is more useful

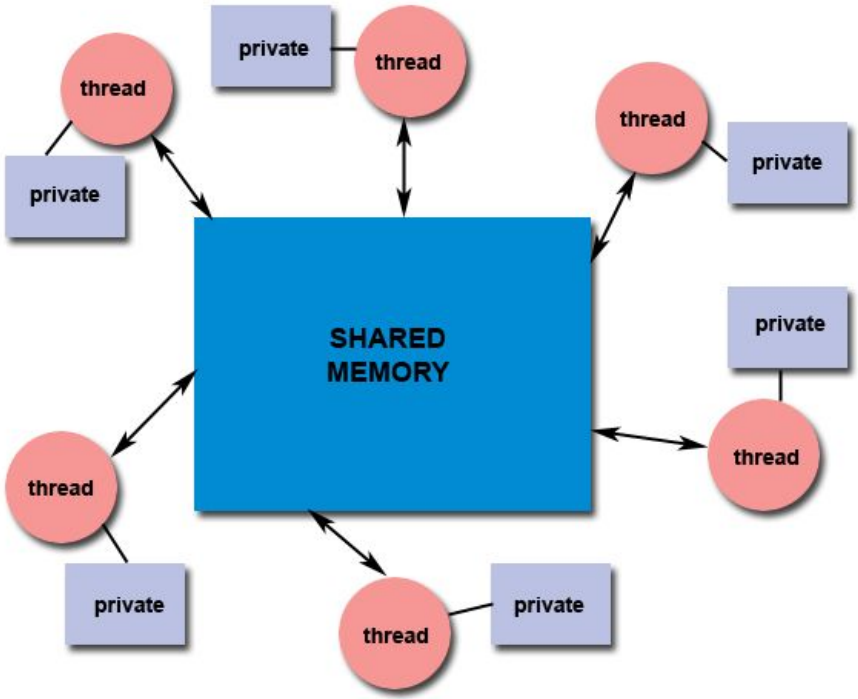
- When you want to leverage the power of a multi-core system to do one task
- When you can’t deal with the overhead of processes
- When you want communication between the processes simplified
- When you want to threads to be part of the same process

Shared memory model

All threads have access to the same global, shared memory

Threads also have their own private data

Programmers are responsible for synchronizing access (protecting) globally shared data.



Review: Creating a thread

```
#include <pthread.h>

int pthread_create(pthread_t *thread,
        const pthread_attr_t *attr,
        void *(*start_routine)(void*),
        void *arg);
```

- On success returns 0
- On error, it returns error code

```
/*example*/
pthread_t *thread;
int r = pthread_create(&thread, NULL, func,
&arg);
if (r != 0)
  perror("pthread_create");
```

pthread_create creates a new thread and makes it executable.

This routine can be called any number of times from anywhere within your code.
pthread_create arguments:

- `thread`: A unique identifier for the new thread returned by the subroutine.
- `attr`: You can specify a thread attributes object, or NULL for the default values.
- `start_routine`: The new thread starts execution by invoking this function
- `arg`: A single argument that may be passed to `start_routine`.
  - It must be passed by reference as a pointer cast of type void.
  - NULL may be used if no argument is to be passed.

Review: Thread termination

The execution of a thread terminates in one of the following ways:

- The thread's start function performs a return specifying a return value for the thread.
- The thread calls `pthread_exit()`
  - `void pthread_exit(void *retval);`
    - Terminates the calling thread
    - `retval` specifies the return value of the thread
    - This value is available to another thread in the same process that calls **pthread_join()**
    - `retval` **should point to the heap not to the stack!**

Note: `exit()` exits the entire process and sets the process' exit value. This is equivalent to `return ();` in the main method.

Issues in shared resources

- Threads can share information via global variables
- Multiple threads should not attempt to modify the same variable at the same time
- Or one thread should not try to read the variable while another thread is modifying it
- **Race condition:** threads are "racing" to access/change the data at the same time
    - At least one thread are modifying(writing) the data
    - The resulting change in data depends on the thread scheduling algorithm
        - the outcome of a program depends on how the kernel schedules the threads (could produce inaccurate results).

```
int count = 0;
void *increment(void *n) {
  int j;
  for (j = 0; j < *((int *)n); j++) {
        count++;
   }
}
```

count++ done in 3 steps:

- Get the current value from **&count**
- Increment by 1
- Store back in **&count**

Race condition: An example bank transaction

Thread 1	Thread 2	Balance
Read balance: $1000		$1000
	Read balance: $1000	$1000
	Deposit $200	$1000
Deposit $200		$1000
Update balance $1000+$200		$1200
	Update balance $1000+$200	$1200

Thread interleaving

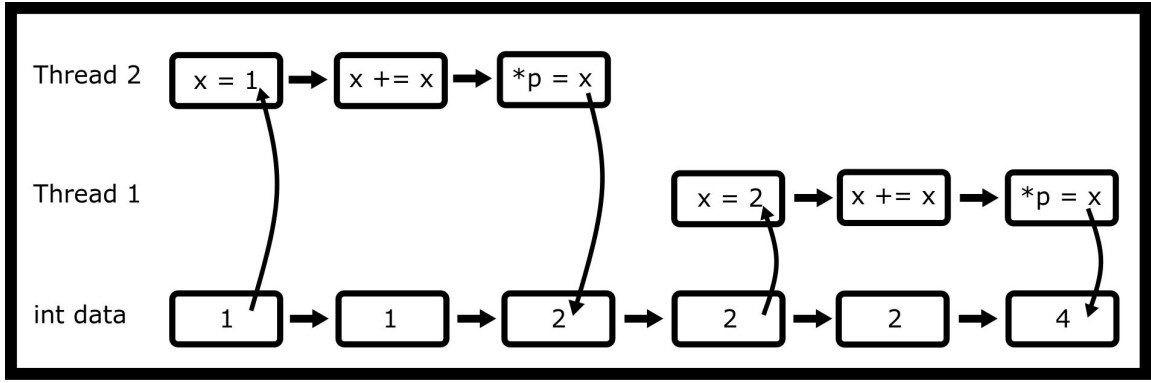
```c
void *thread_main(void *p) {
  int x = *p;
  x += x;
  *p = x;
  return NULL;
}
int main() {
  int data = 1;
  pthread_t one, two;
  pthread_create(&one, NULL, thread_main, &data);
  pthread_create(&two, NULL, thread_main, &data);
  pthread_join(one, NULL);
  pthread_join(two, NULL);
  printf("%d\n", data);
  return 0;
}
```

In assembly, assume data is stored in **eax**

```asm
mov eax, DWORD PTR [rbp-4] ;Loads int_ptr
add eax, eax ;Does the addition
mov DWORD PTR [rbp-4], eax ;Stores it back
```

```c
void *thread_main(void *p) {
  int x = *p;
  x += x;
  *p = x;
  return NULL;
}
int main() {
  int data = 1;
  pthread_t one, two;
  pthread_create(&one, NULL,
      thread_main, &data);
  pthread_create(&two, NULL,
      thread_main, &data);
  pthread_join(one, NULL);
  pthread_join(two, NULL);
  printf("%d\n", data);
  return 0;
}
```

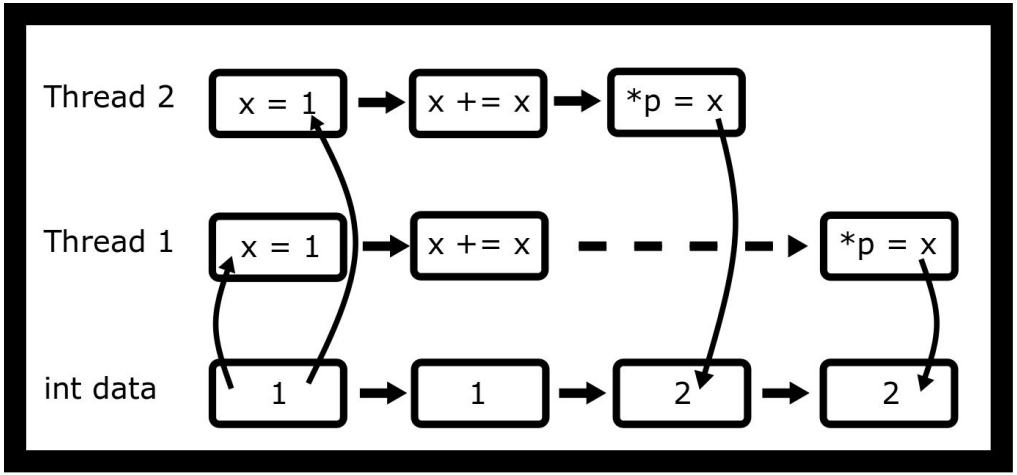
Consider the access pattern



- This access pattern will cause the variable `data` to be 4.
- The problem is when the instructions are executed in parallel (next slide).

```c
void *thread_main(void *p) {
  int x = *p;
  x += x;
  *p = x;
  return NULL;
}
int main() {
  int data = 1;
  pthread_t one, two;
  pthread_create(&one, NULL,
      thread_main, &data);
  pthread_create(&two, NULL,
      thread_main, &data);
  pthread_join(one, NULL);
  pthread_join(two, NULL);
  printf("%d\n", data);
  return 0;
}
```

Consider the access pattern



- This access pattern will cause the variable `data` to be 2.
- This is undefined behavior and a race condition.
- What we want is one thread to access the part of the code at a time.

Thread interleaving

```c
void *thread_main(void *p) {
  int x = *p;
  x += x;
  *p = x;
  return NULL;
}
int main() {
  int data = 1;
  pthread_t one, two;
  pthread_create(&one, NULL, thread_main, &data);
  pthread_create(&two, NULL, thread_main, &data);
  pthread_join(one, NULL);
  pthread_join(two, NULL);
  printf("%d\n", data);
  return 0;
}
```

But when compiled with `-O2`, assembly output is a single instruction.

```
# Optimized way of doing the add
shl dword ptr [rdi]
```

It doesn't fix the problems that the *hardware itself* may experience a race condition because we as programmers didn't tell the hardware to check for it. The easiest way is to add the *lock* prefix (Guide #ref-guide2011intel, 1120).

Race condition example-1: multi threads executing increment()

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define NTHREADS 2
int main(){
  pthread_t tids[NTHREADS];
  int n = 10000;
  for(int i = 0; i < NTHREADS; i++){
       int r = pthread_create(&tids[i],
           NULL, increment, &n);
       if( r != 0){
           perror("thread creation");
           exit(-1);
       }

  }
```

```
  for (int i = 0; i < NTHREADS; i++){
       int r = pthread_join(tids[i], NULL);
       if (r != 0){
           perror("join");
           exit(-1);
       }
  }

  printf("value of count: %d\n", count);
}
```

example-2: passing ref is WRONG!

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
void* myfunc(void* ptr) {
  int i = *((int *) ptr);
  printf("%d \n", i);
  return NULL;
}
int main() {
  /*Each thread gets a different value
 of i to process*/
  int i;
  pthread_t tid;
  for(i =0; i < 10; i++) {
   pthread_create(&tid, NULL, myfunc, &i); //ERROR
  }
  pthread_exit(NULL);
}
```

```c
// correct way, passing value of i
pthread_create(&tid, NULL, myfunc, (void *)i);
```

You may see an output 1 7 8 8 8 8 8 8 8 10

Multi-thread-unsafe function

Some of the provided functions are not thread safe!

E.g.: `asctime`, `getenv`, `strtok`, `strerror`

Programs involving thread-unsafe functions may suffer from a **race condition**

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
char *to_message(int num) {
  static char result [256];
  if (num < 10) sprintf(result, "%d : blah blah" , num);
  else strcpy(result, "Unknown");
  return result;
}
```

Multi-thread-safe function

Instead of making the function responsible for the memory, we made the caller responsible!

A lot of programs, and hopefully your programs, have minimal communication needed.

Often a malloc call is less work than locking a mutex or sending a message to another thread.

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
int to_message_r(int num, char *buf, size_t nbytes){
 size_t written;
 if (num < 10){
   written = snprintf(buf, nbytes, "%d : blah blah", num);
 }else{
   strncpy(buf, "Unknown", nbytes);
   buf[nbytes] = '\0';
   written = strlen(buf) + 1;
 }
 return written <= nbytes;
}
```

Creating thread before fork() results in unexpected errors!

When *fork*() is called, only the calling thread is duplicated in the child process

You can use **pthread_atfork():** However, avoid creating threads before forking unless you fully understand the limitations and difficulties of this approach.

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
static pid_t child = -2;
void *sleepnprint(void *arg) {
  printf("%d:%s starting up...\n", getpid(), (char *) arg);
  while (child == -2) {sleep(1);} /* Later we will use condition
  variables */
  printf("%d:%s finishing...\n",getpid(), (char*)arg);
  return NULL;
}
int main() {
  pthread_t tid1, tid2;
  pthread_create(&tid1,NULL, sleepnprint, "New Thread One");
  pthread_create(&tid2,NULL, sleepnprint, "New Thread Two");
  child = fork();
  printf("%d:%s\n",getpid(), "fork()ing complete");
  sleep(3);
  printf("%d:%s\n",getpid(), "Main thread finished");
  pthread_exit(NULL);
  return 0; /* Never executes */
}
```

Parallel computation: Parallelizing programs

In order to make your program multithread, you need to find concurrent parts of the program

Parallel computation can be done at different levels

- Bit-level parallelism,
- Instruction-level parallelism,
- Data parallelism,
- Task parallelism.

Data parallelism

the same calculation is performed on the same or different sets of data.

- **Sum** of array elements

Task parallelism:

entirely different calculations can be performed on either the same or different sets of data

Embarrassingly Parallel Problems

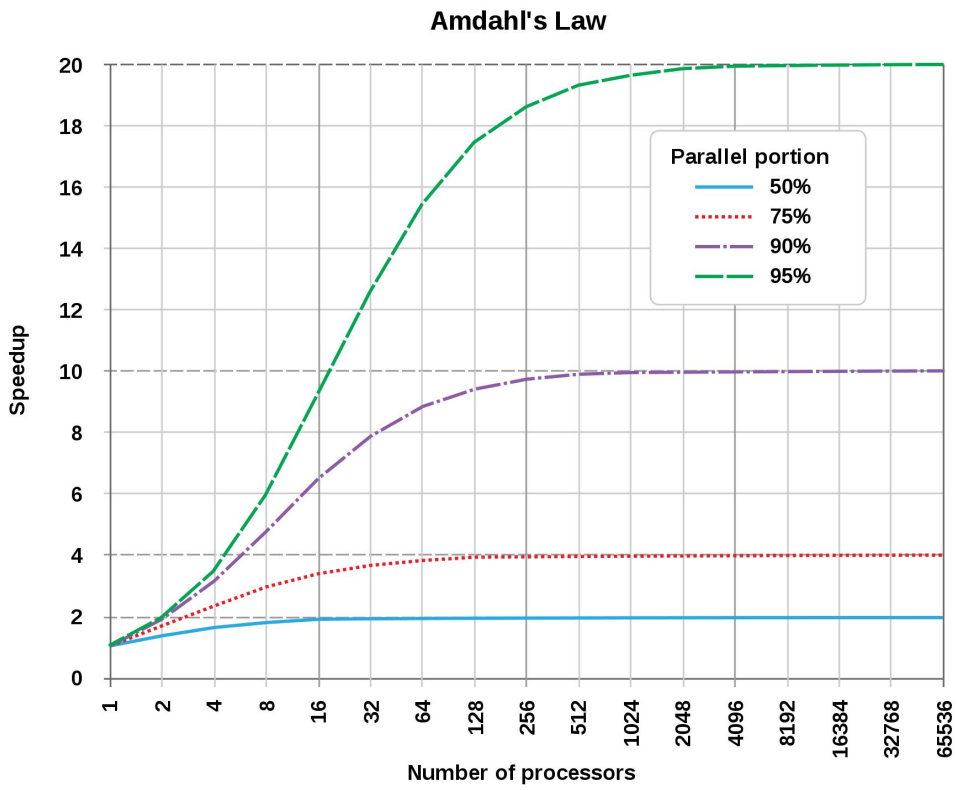
- An embarrassingly parallel problem is any problem that needs little effort to turn parallel.
  - Eg. merge sort
  - Another example is parallel map. Say we want to apply a function to an entire array, one element at a time.
  - https://en.wikipedia.org/wiki/Embarrassingly_parallel

```
void merge_sort(int *arr, size_t
len){
 if(len > 1){
   // Merge Sort the left half
   // Merge Sort the right half
   // Merge the two halves
 }
}
```

```
int *map(int (*func)(int), int *arr, size_t len){
 int *ret = malloc(len*sizeof(*arr));
 for(size_t i = 0; i < len; ++i) {
   ret[i] = func(arr[i]);
 }
 return ret;
}
```

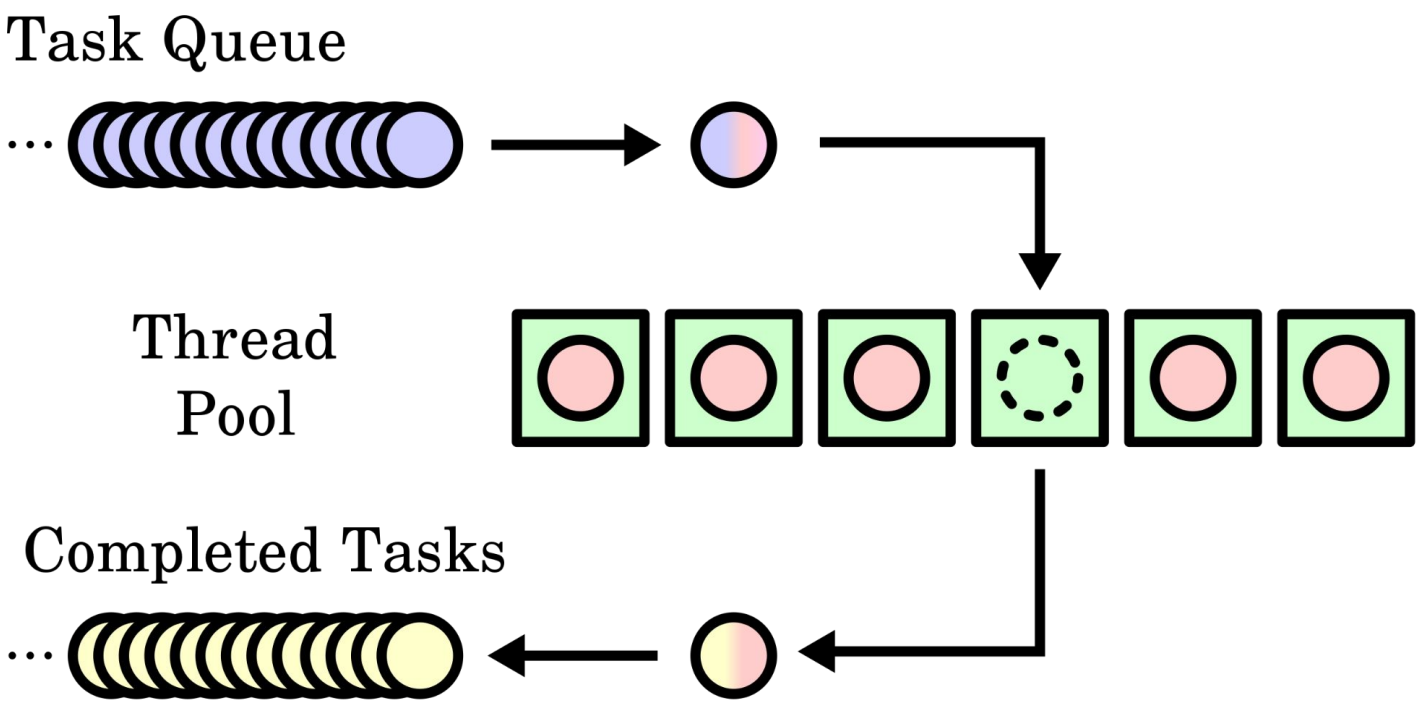
Amdahl's Law

$$S_{\text{latency}}(s) = \frac{1}{(1-p) + \frac{p}{s}}$$



Given that your CPU has multiple real cores, you will see a speedup following https://en.wikipedia.org/wiki/Amdahl's_law.

Thread Pools



How to Implement Thread Pools in C

Task queue:

- A queue of function pointers

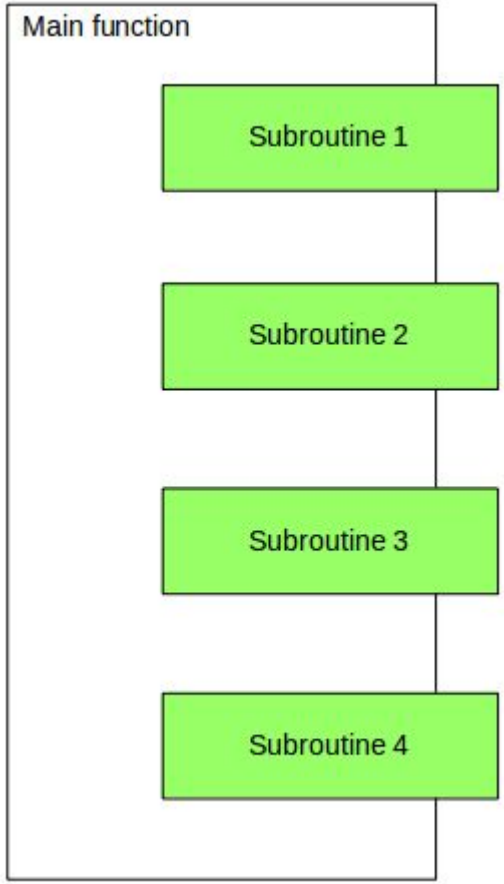
Thread Pool

- A list of threads running a function
  - The function takes the next task from the queue and executes it.
  - If there is no task, it waits for a task
- Need synchronization on thread queue!

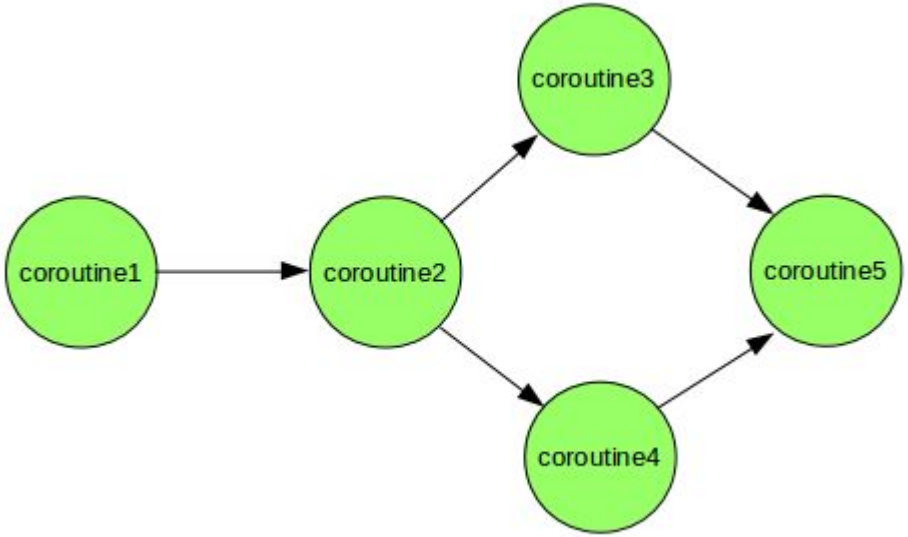
Coroutines (cooperatively multitasked subroutines)

- User level language abstraction (they are stackless)
- subroutines for cooperative multitasking
- Coroutines provide concurrency, because they allow tasks to be performed out of order or in a changeable order, without changing the overall outcome,
- but they do not provide parallelism, because they do not necessarily execute multiple tasks simultaneously.
- Can be used with a multi-threaded dispatcher

https://en.wikipedia.org/wiki/Coroutine



Unlike standard subroutines, program with coroutines has multiple **entry** point



https://www.geeksforgeeks.org/coroutine-in-python

Python example

How to implement this in C?

```python
def print_name(prefix):
    print("Searching prefix:{}".format(prefix))
    while True:
        name = (yield)
        if prefix in name:
            print(name)
```

```python
 calling coroutine, nothing will happen
corou = print_name("Dear")

# This will start execution of coroutine and
# Prints first line "Searching prefix..."
# and advance execution to the first yield expression
corou.__next__()

# sending inputs
corou.send("Atul")
corou.send("Dear Atul")
corou.close()
```

https://www.geeksforgeeks.org/coroutine-in-python/

Goroutines (Green threads)

- Go language user level thread routines
  - Thread scheduler at the user level managed by Go runtime)
  - Stackless
  -

```go
package main

import (
	"fmt"
	"time"
)

func say(s string) {
	for i := 0; i < 5; i++ {
		time.Sleep(100 * time.Millisecond)
		fmt.Println(s)
	}
}
```

```go
func main() {
	go say("world") // new goroutine
	say("hello")
}
```

Next

Multi-thread and process synchronization